

Ejercicio 3

Diga qué función de la clase punto se llama en cada una de las siguientes líneas. Si alguna depende de una línea anterior que sea incorrecta, corrijala previamente.

```
1  class punto {
2      double x, y;
3
4  public:
5      punto(double a = 0., double b = 0.) : x{a}, y{b} {}
6
7      punto(const punto& p) : x{p.x}, y{p.y} {}
8
9      punto& operator=(const punto& p)
10     {
11         x = p.x;
12         y = p.y;
13         return *this;
14     }
15 };
```

C++

```
1  punto p;
2  punto q();
3  punto r(2.,);
4  punto s{3.4};
5
6  punto t{};
7  punto u(q);
8  punto v = r;
9  t = s;
```

C++

1. Llama al ctor. por defecto punto(double a = 0., double b = 0.)
2. No es válida la declaración. Opción cambio : punto q = punto()
3. Antes debemos quitar la coma después de 2. y llama al ctor. de punto con el valor b por defecto, es decir, punto(double a = 2., double b = 0)
4. Llama al ctor. de punto con el valor b por defecto, es decir, punto(double a = 3.4, double b = 0)
6. Llama al ctor. por defecto punto(double a = 0., double b = 0.)
7. Llama al ctor. de copia punto(const punto&)
8. Llama al ctor de copia con r como parámetro
9. Llama al operador de asignación.

Ejercicio 4

Decida si X se puede sustituir por 1, 2, ambos o ninguno en los siguientes items:

- 1 Se puede definir: LibroX lib1;
- 2 Se tiene un constructor de conversión de std::string a LibroX
- 3 Se puede definir: LibroX lib2[5];
- 4 Se puede definir: std::vector lib3;
- 5 Siendo "El Quijote" una cadena literal de tipo const char*, se produce una conversión implícita a string al ejecutar: LibroX* lib4 = new LibroX("El Quijote");
- 6 Se puede definir: LibroX lib5 = "El Quijote";
- 7 Hace falta definir el destructor para LibroX.

```
1  class Libro1 {
2      string titulo_;
3      int pags_;
4
5  public:
6      Libro1(string t = "", int p = 0);
7      // ...
8  };
9
10 class Libro2 {
11     string titulo_;
12     int pags_;
13
14 public:
15     Libro2(string t, int p = 0);
16     Libro2(const char* c);
17     // ...
18 };
```

1. Se puede definir Libro1 nada más porque Libro2 no tiene constructor por defecto.
2. Libro1 y Libro2 si disponen de constructor de conversión de std::string
3. Sólo se podría hacer para Libro1 ya que necesitaríamos constructor por defecto y Libro2 no dispone de él
4. Se podría hacer para Libro1 y Libro2 ya que la clase vector aún no llama a ningún ctor.
5. Se podría hacer perfectamente para Libro1 y para Libro2 no porque si se produce una conversión a std::string Libro2 no dispone de constructor a partir de std::string.
6. Se podría definir para Libro2 ya que posee constructor de conversión de const char*.
7. No hace falta generar destructor ya que el compilador generará un ctor que llamara al destructor de cada atributo y en este caso esos atributos poseen correctamente sus destructores.

Ejercicio 5

Diga si el siguiente programa funciona correctamente. En caso afirmativo indique lo que imprime. En caso negativo haga las modificaciones necesarias para que funcione correctamente.

```
1  #include <iostream>
2  #include <cstring>
3
4  using namespace std;
5
6  class Libro {
7      char* titulo_;
8      int paginas_;
9
10 public:
11     Libro() : titulo_(new char[1]{0}), paginas_(0) {}
12
13     Libro(const char* t, int p) :
14         titulo_(new char[strlen(t) + 1]),
15         paginas_(p)
16     {
17         strcpy(titulo_, t);
18     }
19
20     ~Libro()
21     {
22         delete[] titulo_;
23     }
24
25     int paginas() const
26     {
27         return paginas_;
28     }
29
30     char* titulo() const
31     {
32         return titulo_;
33     }
34 };
35
36 void mostrar(Libro l)
37 {
38     cout << l.titulo() << " tiene "
39         << l.paginas() << " paginas" << endl;
40 }
41
42 int main()
43 {
44     Libro l1("Fundamentos de C++", 474), l2;
```

C++

```

45
46     l2 = l1;
47
48     mostrar(l1);
49     mostrar(l2);
50 }

```

Principalmente problema es que no existen ctor. de copia ni operador de asignación por copia. Por tanto l2.titulo apuntara a l1.titulo

mostrar(l1) no haría bien la copia y el parametro l.titulo recibiría la dirección de memoria de l1.titulo imprimiría todo correctamente y al salir destruiría l y por tanto libera l1.titulo también.

mostrar(l2) fallaría completamente porque l2.titulo apunta a l1.titulo que ha sido liberado anteriormente.

Imprimiría: "Basura" tiene 474 páginas.

La solución es crear un ctor. de copia y un operador de asignación por copia.

```

1  //Añadimos ctor. copia
2
3  Libro::Libro(const Libro& l) : Libro(l.titulo_, l.paginas_) {}
4
5  Libro& Libro::operator =(const Libro& l)
6  {
7      Libro copia(l);
8      std::swap(titulo_, copia.titulo_);
9      std::swap(paginas_, copia.paginas_);
10
11     return *this;
12 }

```

 C++

Ejercicio 6

Describe el error de compilación que provoca el código anterior. ¿Cómo modificaría las clases sin suprimir métodos para solucionarlo?

Suponga que el parámetro de f() es de entrada y salida y la línea es sustituida por void f(A&). ¿Qué error de compilación se produce? ¿Y cómo se puede resolver sin suprimir métodos?

```
1  struct B; //Declaración adelantada
2  struct A
3  {
4      A(B);
5  };
6
7  struct B
8  {
9      operator A();
10 };
11
12 void f(const A&);
13
14 int main()
15 {
16     B bf; f(b);
17 }
```

1. El problema está en que no sabe si usar el operador de conversión o el constructor de conversión, por tanto, hay una ambigüedad. La solución es hacer uno de los dos métodos explícit.
2. El problema sería que a A& estaríamos pasándole un objeto temporal y esto sólo está permitido para const A&

Solución

```
1  int main()
2  {
3      B b;
4
5      A a(b);
6      f(b);
7  }
```